

**COMSATS UNIVERSITY ISLAMABAD,
ATTOCK CAMPUS**



SUBMITTED TO :

Mr. Shahzad Rizwan
Advanced Database Systems

SUBMITTED BY:

Maryam Yaseen Fa23-bcs-058
Aneeba Hassan Fa23-bcs-030
Momina Ishfaq Fa23-bcs-060
Ayesha Ahmad Fa23-bcs-018
Maham Iftikhar Fa23-bcs-009

Advanced Database Systems Project Report

AI-Powered Learning Management System (LMS)

Full-Stack Online Education Platform with Role-Based Access, Course Management, MongoDB Data Modeling and Future AI Enhancements

AI-Powered LMS - System Architecture

Layered architecture for React frontend, Express APIs, MongoDB database and future AI modules.

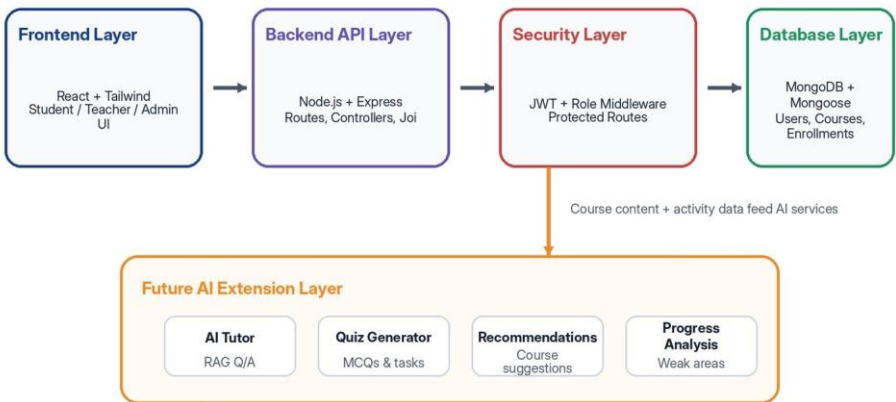


Figure 1: High-level architecture of the proposed AI-powered LMS.

Table of Contents

1. Executive Summary
2. Introduction and Problem Statement
3. Project Objectives
4. Scope of the System
5. Technology Stack
6. System Architecture
7. User Roles and Functional Requirements
8. Database Design and MongoDB Collections
9. Mongoose Schemas
10. Backend API Design
11. Authentication and Authorization
12. Frontend Design
13. Admin Panel
14. Advanced Database Concepts Used
15. Validation, Error Handling and Security
16. Future AI Features
17. Testing Strategy
18. Deployment Plan
19. Limitations
20. Conclusion

1. Executive Summary

This document presents a detailed Advanced Database Systems project report for an AI-Powered Learning Management System (LMS). The proposed system is a full-stack web application designed for students, teachers and administrators. It allows teachers to create courses, students to enroll in courses, and administrators to manage platform users, courses and analytics. The system is designed around a MongoDB database using Mongoose schemas, RESTful APIs, JWT authentication, and role-based access control.

The project is suitable for an advanced database course because it includes real-world database modeling, document schema design, relationships using ObjectId references, indexing requirements, CRUD operations, validation, aggregation-based analytics, data security, and future AI-ready database extensions. The system also provides a strong foundation for semantic search, recommendation systems, AI tutors and RAG-based educational assistants.

2. Introduction and Problem Statement

Online learning platforms are now widely used by universities, schools and training organizations. However, many basic LMS applications only provide static course pages and do not focus on flexible role-based management, dynamic database-driven content, or AI-ready architecture. Students need an easy way to browse and enroll in courses. Teachers need a complete panel to create and manage learning content. Administrators need visibility and control over users, content quality and platform activity.

The main problem addressed by this project is the design and implementation of a scalable, database-driven LMS that can manage multiple user roles, courses, enrollments and future AI services. The system must support secure authentication, efficient storage of course data, protected role-based dashboards, and clean APIs for frontend-backend communication.

3. Project Objectives

- Develop a full-stack LMS web application using ReactJS, Tailwind CSS, Node.js, Express.js and MongoDB.
- Implement secure user registration and login using JWT-based authentication and hashed passwords.
- Support three major roles: Student, Teacher and Admin, each with a separate dashboard and permissions.
- Design MongoDB collections for users, courses, enrollments, categories, content and activity logs.
- Implement course CRUD operations where teachers can create, update, delete and manage courses.
- Allow students to browse courses, view details, enroll and manage enrolled courses.
- Allow administrators to manage users, manage courses, monitor analytics and control platform settings.
- Use Joi validation and centralized error handling to improve backend reliability.
- Prepare the system architecture for future AI features such as chatbot tutor, recommendations and AI- generated quizzes.

4. Scope of the System

The project scope covers the main academic LMS workflow from user authentication to course management and enrollment. It is designed as a database project, so special focus is placed on data organization, relationships, validation, scalability and maintainability. The first version will not require a fully deployed AI model, but the architecture will include placeholders and database support for future AI

modules.

In-Scope Features

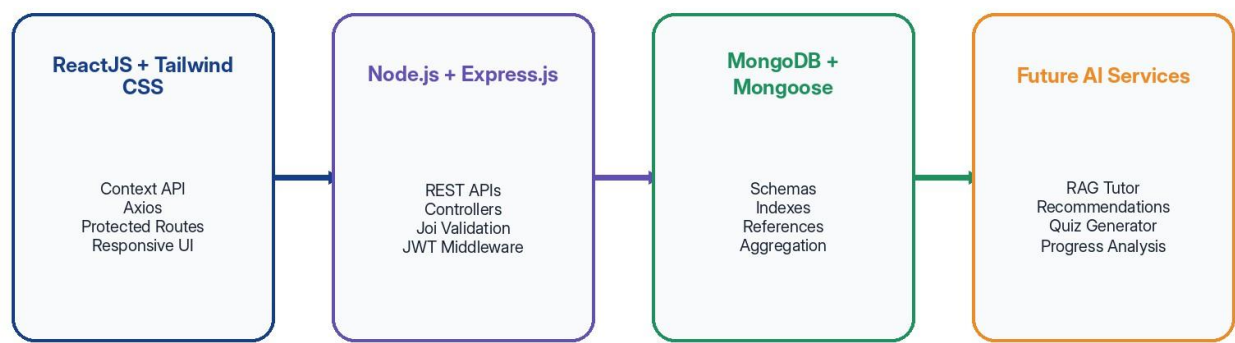
- Student signup/login and dashboard
- Teacher signup/login and dashboard
- Admin dashboard for platform management
- Course listing, course detail and enrollment flow
- Teacher course creation, editing and deletion
- MongoDB/Mongoose schemas and RESTful APIs
- Role-based protected routes and JWT middleware
- Basic analytics through database aggregation

Out-of-Scope for Initial Version

- Live video conferencing integration
- Payment gateway integration for paid courses
- Production-grade AI deployment
- Mobile app version
- Full enterprise-level audit compliance

5. Technology Stack

Technology Stack and Data Flow



Example data flow: User action → Axios request → Express route → auth/role middleware → controller/service → Mongoose model → MongoDB response → React state up

Figure 2: Technology stack and end-to-end data flow.

Layer	Technology	Purpose
Frontend	ReactJS	Builds reusable UI components and role-based dashboards.
Styling	Tailwind CSS	Provides modern, responsive and utility-first styling.
State Management	React Context API	Stores authenticated user, token and global app state.

API Client	Axios	Sends HTTP requests from frontend to backend APIs.
Backend	Node.js + Express.js	Implements REST APIs, routing, middleware and controllers.
Database	MongoDB	Stores users, courses, enrollments and activity logs as documents.
ODM	Mongoose	Defines schemas, models, validation rules and references.
Authentication	JWT + Local Storage	Maintains user sessions and protected routes.
Validation	Joi	Validates request body before database operations.

6. System Architecture

The architecture follows a three-layer model with a future AI extension layer. The frontend layer contains role- specific dashboards. The backend layer contains REST APIs, authentication middleware, validation and business logic. The database layer stores all persistent data in MongoDB collections. The AI layer is planned for future modules such as RAG tutoring, recommendations and progress analysis.

AI-Powered LMS - System Architecture

Layered architecture for React frontend, Express APIs, MongoDB database and future AI modules.

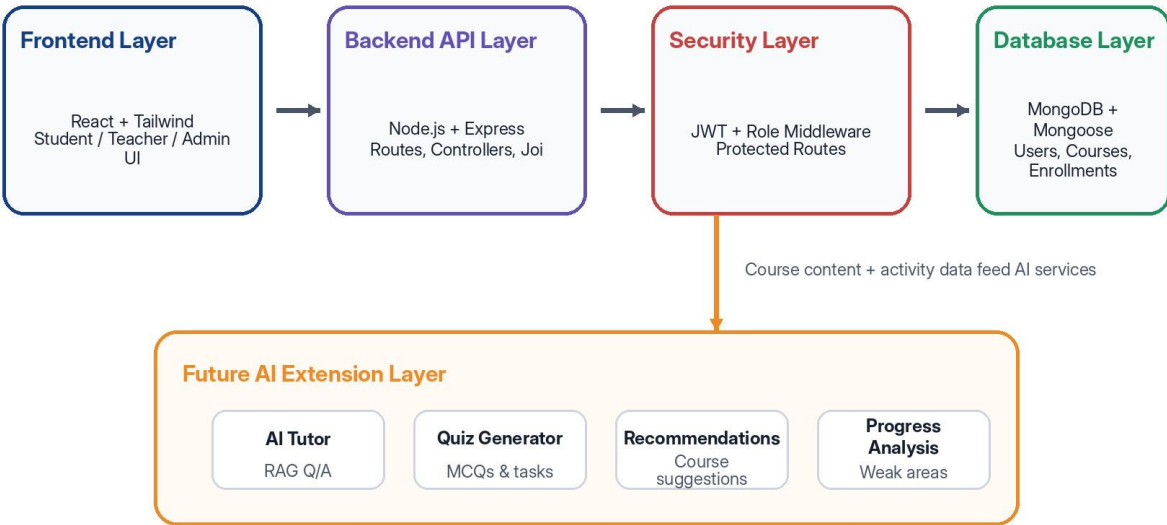


Figure 3: Full system architecture with frontend, backend, database and AI extension layer.

Architecture Explanation

- Client Layer: React pages are separated into public pages, student pages, teacher pages and admin pages.
- API Layer: Express routes receive requests and pass them through validation, authentication and controller logic.

- Security Layer: JWT middleware verifies tokens and role middleware restricts access according to user type.
- Database Layer: MongoDB stores normalized document collections with references such as teacher-to- course and student-to-enrollment.
- AI Extension Layer: Future AI services can read course content, student activity and learning data to generate intelligent responses.

7. User Roles and Functional Requirements

Role-Based Workflow

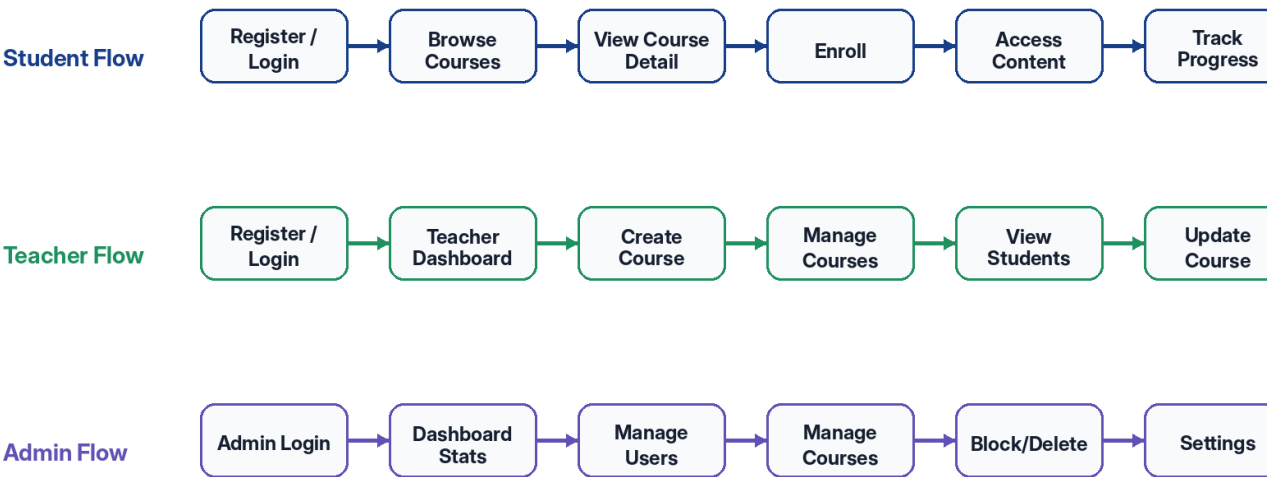


Figure 4: Role-based workflow for student, teacher and admin users.

7.1 Student Panel

The student panel is focused on learning and enrollment. Students can create accounts, log in securely, browse all available courses, open a detailed course page and enroll in the course. After enrollment, the student can access learning content, view course objectives, read welcome messages and manage profile information.

Student Feature	Detailed Description
Signup and Login	Students register using name, email and password. On login, the backend returns a JWT token and user role.
Browse Courses	Students can view all active and approved courses from the database.
Course Details	Students can see title, subtitle, category, price, level, language, objectives and welcome message.
Enroll in Course	Enrollment creates a relation between the student and the course in the enrollments collection.
Enrolled Courses	Student dashboard shows only courses in which the logged-in student is enrolled.
Profile Update	Students can edit basic profile information such as name and preferences.

7.2 Teacher Panel

The teacher panel is focused on course creation and content management. Teachers can add full course details including title, category, code, price, description, image, objectives and welcome message. Teachers can also update existing courses, delete courses and view students enrolled in their courses.

Teacher Feature	Detailed Description
Create Course	Teacher submits course data through a protected form. Backend validates the request and stores it in MongoDB.
Manage Courses	Teacher can view only courses created by their own user ID.
Edit Course	Teacher can update course metadata, content, image and objectives.
Delete Course	Teacher can delete or deactivate courses they own.
View Enrolled Students	Teacher can see students enrolled in their courses using enrollment lookup.
Upload Lectures	Future content module allows lecture files, notes or links to be attached to a course.

7.3 Admin Panel

The admin panel gives central control over the platform. Admins can manage users, manage courses, block or delete accounts, approve courses and monitor activity. Admin dashboards are important because they help maintain content quality and platform security.

Admin Feature	Detailed Description
View Users	Admin can see all students, teachers and administrators.
Block/Delete Users	Admin can block suspicious accounts or remove invalid users.
Manage Courses	Admin can view, approve, reject or delete any course.
Dashboard Statistics	Admin can see total users, courses, enrollments and recent activity.
Manage Categories	Admin can add or deactivate course categories.
Platform Settings	Admin can configure general LMS settings in future versions.

8. Database Design and MongoDB Collections

MongoDB is selected because LMS data is naturally document-oriented. Course records contain nested arrays such as objectives, metadata and content references. Mongoose provides schema-level structure, validation and references between collections. The database design uses a hybrid approach: embedded arrays for small course details and referenced collections for users, enrollments and activity logs.

MongoDB Logical Data Model / ERD

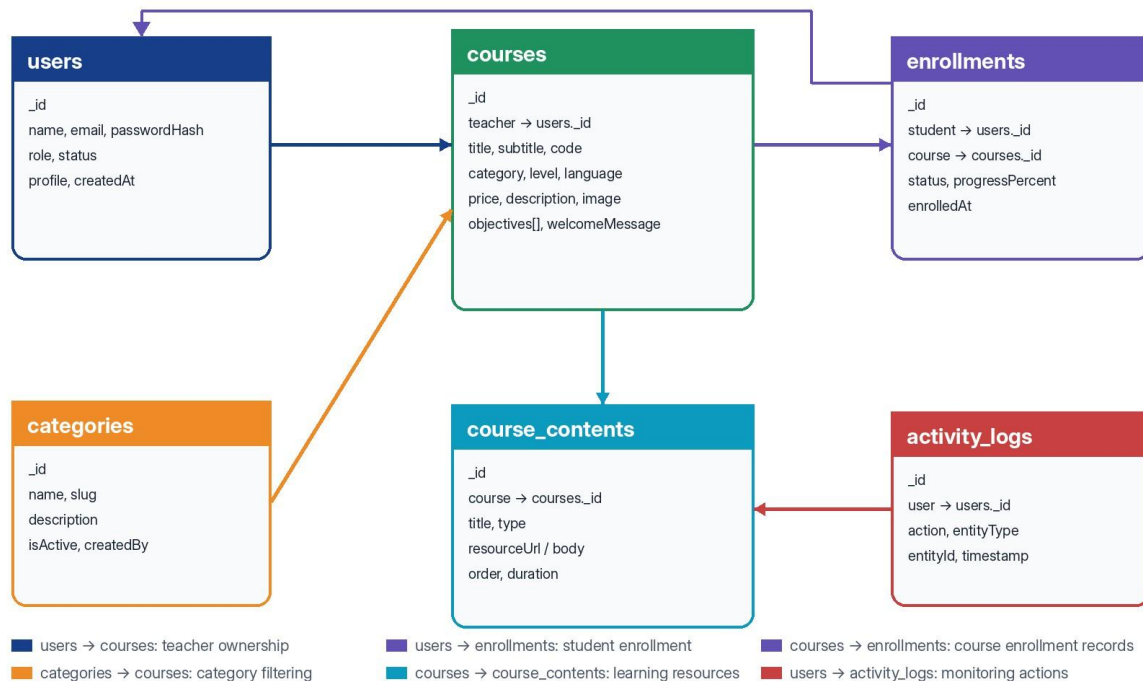


Figure 5: Logical ERD showing main MongoDB collections and relationships.

8.1 Main Collections

Collection	Purpose	Important Fields
users	Stores all platform users with role-based identity.	name, email, passwordHash, role, status, profile, createdAt
courses	Stores complete course information created by teachers.	teacher, title, subtitle, category, code, language, level, price, description, objectives, welcomeMessage, image
enrollments	Stores student-course relationship and progress.	student, course, status, enrolledAt, progressPercent
categories	Stores course categories managed by admin.	name, slug, description, isActive
course_contents	Stores lecture or resource records for courses.	course, title, type, resourceUrl, order, duration
activity_logs	Stores platform activity for analytics and monitoring.	user, action, entityType, entityId, metadata, timestamp

8.2 Course Schema

The course collection is the core collection of the LMS. Each course belongs to a teacher and contains all data needed for display, management and future AI processing.

Field	Type	Description
user / teacher	ObjectId ref User	The teacher who created the course.
title	String	Main course title.

subtitle	String	Short supporting line displayed under the title.
category	String / ObjectId	Course category such as Computer Science, Mathematics or AI.
code	String	Unique course code such as CS401 or DBMS-202.
language	String	Course language such as English or Urdu.
level	String	Beginner, Intermediate or Advanced.
price	Number	Course price; can be zero for free courses.
description	String	Detailed course overview.
objectives	Array of Strings	Learning outcomes of the course.
welcomeMessage	String	Introductory message shown to enrolled students.
image	String	Course thumbnail URL or uploaded image path.
dateCreated	Date	Creation timestamp.

9. Mongoose Schema Design

Below is the conceptual schema design that can be implemented in the backend. In the actual project, each schema should be placed inside the models folder and exported as a Mongoose model.

9.1 User Model

The user model stores students, teachers and admins in one collection. A single collection simplifies authentication and role-based access. The role field determines which dashboard and APIs the user can access.

Field	Validation / Rule
name	Required string, minimum 2 characters
email	Required, unique, lowercase, valid email
passwordHash	Required hashed password using bcrypt
role	Enum: student, teacher, admin
status	Enum: active, blocked
profile	Optional object for role-specific details
timestamps	createdAt and updatedAt enabled

9.2 Course Model

The course model contains a reference to the teacher. This makes it possible to query all courses created by a teacher and also show teacher details using population.

Design Rule	Reason
teacher field uses ObjectId reference to users	Maintains ownership and supports teacher-specific course management.
objectives are stored as an array	Allows multiple learning outcomes inside one course document.

category can be string initially and ObjectId later	Supports quick implementation and future category management.
timestamps are enabled	Supports sorting by latest courses and audit history.
indexes on title, category and teacher	Improves search and dashboard performance.

10. Backend API Design

The backend follows RESTful API principles. Routes are divided into authentication routes, user routes, course routes, enrollment routes and admin routes. Each route should call middleware for authentication and role checking before reaching the controller.

Module	Endpoint Example	Access	Purpose
Auth	POST /api/auth/register	Public	Register a new student, teacher or admin depending on allowed policy.
Auth	POST /api/auth/login	Public	Authenticate user and return JWT token.
Users	GET /api/users/me	Authenticated	Return logged-in user profile.
Courses	GET /api/courses	Public / Authenticated	Fetch all active courses.
Courses	GET /api/courses/:id	Public / Authenticated	Fetch course details by ID.
Courses	POST /api/courses	Teacher	Create a new course.
Courses	PUT /api/courses/:id	Teacher / Admin	Update course details.
Courses	DELETE /api/courses/:id	Teacher / Admin	Delete or deactivate a course.
Enrollments	POST /api/enrollments/:courseId	Student	Enroll logged-in student in a course.
Admin	GET /api/admin/stats	Admin	Return dashboard statistics using aggregation.
Admin	GET /api/admin/users	Admin	Fetch all platform users.
Admin	PATCH /api/admin/users/:id/block	Admin	Block or unblock a user.

11. Authentication and Authorization

Authentication verifies the identity of the user, while authorization verifies what the user is allowed to do. The LMS uses JWT tokens. After successful login, the backend signs a token containing user ID and role. The frontend stores the token in local storage and sends it in the Authorization header for protected requests.

Security Flow

- User submits email and password from login page.
- Backend finds the user by email.

- Password is compared with bcrypt hash.
- JWT token is generated with user ID and role.
- Frontend stores token and user data in local storage or context state.
- Protected routes check token before allowing dashboard access.
- Role middleware prevents students from accessing teacher/admin APIs.

Role-Based Access Matrix

Feature	Student	Teacher	Admin
Browse Courses	Yes	Yes	Yes
Enroll in Course	Yes	No	No
Create Course	No	Yes	No
Edit Own Course	No	Yes	No
Delete Any Course	No	No	Yes
View All Users	No	No	Yes
Block Users	No	No	Yes
View Platform Analytics	No	Limited	Yes

12. Frontend Design

The frontend should be clean, responsive and role-based. React Router can be used for navigation, while Context API stores authentication state. Tailwind CSS should be used for styling dashboards, forms, cards and tables. Axios should be configured with a base URL and token interceptor.

Suggested Dashboard UI Screens

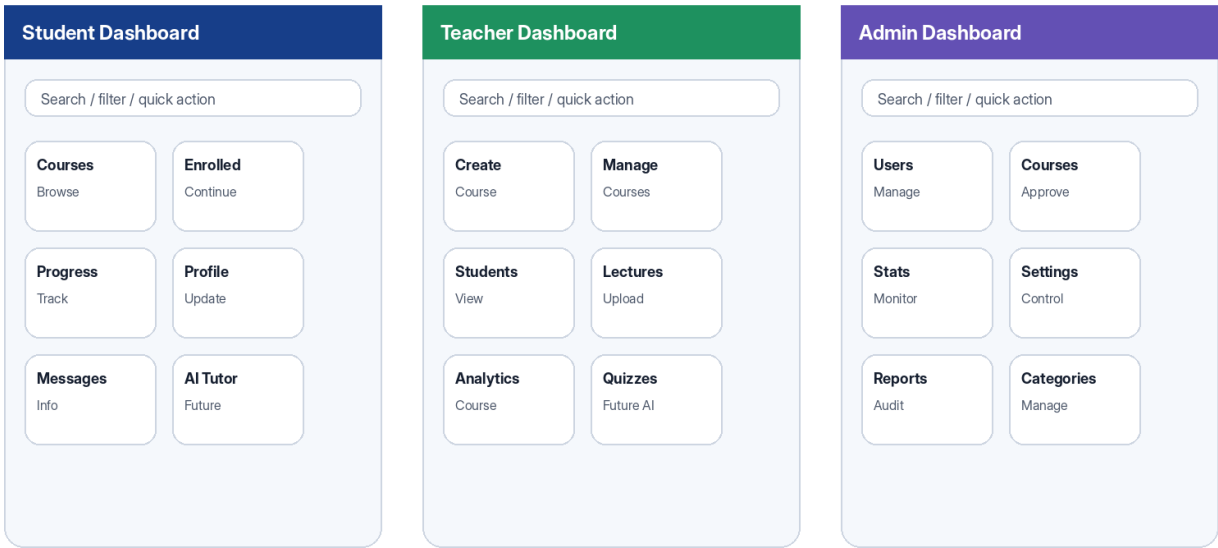


Figure 6: Suggested visual dashboard layout for student, teacher and admin panels.

Frontend Pages

Page Type	Pages	Purpose
-----------	-------	---------

Public Pages	Landing, Login, Signup	Introduce platform and allow account access.
Student Pages	Dashboard, Courses, Course Detail, Enrolled Courses	Allow students to browse, enroll and learn.
Teacher Pages	Dashboard, Create Course, Manage Courses, Edit Course	Allow teachers to manage course content.
Admin Pages	Dashboard, Manage Users, Manage Courses, Analytics	Allow platform administration and monitoring.

13. Admin Panel and Analytics

The admin panel is important for database visibility. It can use MongoDB aggregation pipelines to calculate statistics such as total users, number of students, number of teachers, total courses, enrollments per course and recent activity. These analytics show that the project uses advanced database operations beyond simple CRUD.

Statistic	Database Method
Total users by role	Group users by role using aggregation.
Total courses by category	Group courses by category.
Most enrolled courses	Lookup enrollments and sort by count.
Recent activity	Sort activity_logs by timestamp.
Blocked users count	Filter users where status = blocked.
Teacher course count	Group courses by teacher.

14. Advanced Database Concepts Used

Because this is an Advanced Database Systems project, the implementation should clearly demonstrate database concepts. MongoDB is not only used as a simple storage layer; it supports schema design, indexing, aggregation, references, validation and optimized queries.

Concept	How It Is Applied in LMS
Document Modeling	Course data is stored as rich documents with nested objectives and metadata.
Referencing	Courses reference teachers; enrollments reference both students and courses.
Indexing	Indexes on email, course title, category, teacher and createdAt improve query speed.
Aggregation	Admin analytics use grouping, counting, lookup and sorting.
Validation	Mongoose and Joi validation prevent invalid data from entering the database.
Concurrency Control	Duplicate enrollments are prevented using unique compound indexes on student + course.

Security	Passwords are hashed and JWT tokens protect private APIs.
Scalability	Collections can be extended for lectures, quizzes, progress and AI features.

Recommended Indexes

Collection	Index	Reason
users	unique index on email	Prevents duplicate accounts and speeds up login.
courses	index on teacher	Speeds teacher dashboard course loading.
courses	text index on title and description	Supports course search.
courses	index on category and level	Speeds filtering.
enrollments	unique compound index on student + course	Prevents duplicate enrollment.
activity_logs	index on timestamp	Speeds admin activity feed.

15. Validation, Error Handling and Security

A professional backend must handle invalid input and unexpected errors. Joi validation should be applied before controller logic. A centralized error handler should return consistent JSON responses. Security should include password hashing, protected routes, role checking and safe handling of environment variables.

- Use bcrypt for password hashing instead of storing plain passwords.
- Use JWT secret from environment variables, not hard-coded values.
- Use Joi schemas for registration, login and course creation forms.
- Use global error middleware for clean error responses.
- Validate MongoDB ObjectId before database queries.
- Prevent unauthorized course editing by checking teacher ownership.
- Do not expose passwordHash in API responses.
- Use CORS configuration for frontend-backend connection.

16. Future AI Features

The LMS is designed to support future AI features. These features can be integrated after the core database application is stable. The database should store course content, student progress and activity logs so AI services can use this data for personalized learning.

AI Feature	Description	Database Support Needed
AI Chatbot Tutor	Answers student questions based on course content.	Course content, lecture text, vector embeddings, chat history.
Personalized Recommendations	Suggests courses based on role, activity, category and progress.	Enrollments, progress records, course categories and user preferences.

AI-Generated Quizzes	Creates quizzes from objectives and lecture content.	Course content, quiz collection, question bank.
Semantic Course Search	Allows natural language search instead of keyword-only search.	Course embeddings and vector index.
RAG Educational Assistant	Retrieves relevant course material and generates grounded answers.	Document chunks, embeddings, source references.
AI Study Planner	Creates weekly study plans based on student progress.	Progress records, deadlines, course difficulty.
Progress Analysis	Detects weak areas and predicts performance risks.	Quiz attempts, activity logs and completion data.

17. Testing Strategy

Testing should verify both frontend and backend behavior. Since this is a database project, database operations must be tested carefully to ensure that relationships, validation and permissions work correctly.

Test Type	What to Test
Authentication Testing	Register, login, invalid password, duplicate email and protected route access.
Role Testing	Student cannot access teacher/admin routes; teacher cannot access admin routes.
Course CRUD Testing	Create, read, update and delete course with valid and invalid data.
Enrollment Testing	Student can enroll once; duplicate enrollment is blocked.
Admin Testing	Admin can view users, block users and delete inappropriate courses.
Database Testing	Indexes, references, population and aggregation queries.
Frontend Testing	Responsive layout, route protection and API error display.

18. Deployment Plan

The application can be deployed using separate frontend and backend services. The backend should use environment variables for database URI, JWT secret and port configuration. MongoDB Atlas can be used as the cloud database. The frontend can be deployed on Vercel or Netlify, while the backend can be deployed on Render, Railway or similar Node.js hosting platforms.

- Create MongoDB Atlas cluster and obtain connection URI.
- Configure backend environment variables: MONGO_URI, JWT_SECRET, PORT and CLIENT_URL.
- Deploy Express backend and test API health endpoint.
- Deploy React frontend and set backend API base URL.

- Enable CORS for deployed frontend domain.
- Test login, dashboard and course CRUD after deployment.

19. Limitations

The initial version focuses on database-driven LMS functionality. Some features are planned for future expansion. These limitations are acceptable for the first version because the main goal is to demonstrate advanced database design and full-stack implementation.

- AI features are designed architecturally but may not be fully implemented in the first version.
- Payment integration is not included in the core implementation.
- File upload may initially use image URLs instead of cloud storage.
- Course progress tracking can start with simple percentage values before advanced analytics.
- Admin analytics may initially be basic and can be expanded with more aggregation reports.

20. Conclusion

The AI-Powered LMS is a strong Advanced Database Systems project because it combines real-world full-stack development with meaningful database design. It includes multiple user roles, course management, enrollments, admin controls, secure authentication, and MongoDB-based modeling. The proposed architecture is also future-ready for AI features such as chatbot tutoring, semantic search, personalized recommendations and AI-generated quizzes.

By implementing this project, the group will demonstrate practical skills in database schema design, API development, role-based security, frontend integration, analytics and scalable software architecture. The project can be extended further into a production-ready learning platform after completion of the core modules.